International Journal of Control, Automation, and Systems VV(X) (YYYY) 1-11
http://dx.doi.org/10.1007/s12555-xxx-xxxx-x
ISSN:1598-6446 eISSN:2005-4092
http://www.springer.com/12555

Intent-Guided Action Representation Learning for Dexterous Hand Control in Vision-Language-Action Models

Minchan Jin†, Geunha Kim†, and Jun Moon*

Abstract: Vision-Language-Action models (VLAs) offer a unified interface for mapping visual observations and language instructions to robot actions, but dexterous hands remain challenging because multi-finger actions are high-dimensional, strongly coupled, and sensitive to contact timing. We propose a VLA-based arm–hand pipeline with two complementary modules. Hand Synergy Latent represents 16-dimensional hand commands in a shared four-dimensional PCA space, and a fixed decoder reconstructs 16-dimensional joint commands for execution. Hand Intent predicts target hand progress and selects a hand action from the VLA-predicted action chunk. Across six seeds, PCA-K4 reaches the normalized 90% and 95% learning criteria in 13.5% and 28.2% fewer optimization steps than Direct16, respectively. Hand Intent reduces both alignment errors across all tasks and yields complete outcomes in every nominal-position trial. For pull, fixed selection yields two complete and eight partial outcomes, whereas Hand Intent yields ten complete outcomes.

Keywords: Dexterous Manipulation, Robot Learning, Vision-Language-Action Model.

1. INTRODUCTION

Vision-Language-Action models (VLAs) [1–8] provide a promising interface for robot manipulation by connecting visual observations and language instructions to robot actions. Generalist policies motivate reusable image–language–action architectures across embodiments and tasks [9–11].

Integrating dexterous manipulation into this formulation remains challenging. A dexterous hand must coordinate multiple joints, form task-specific postures, and transition at the appropriate moment relative to object contact. Prior studies show the difficulty of high-dimensional multi-finger control and contact-rich interaction [12–15]. Some dexterous VLA systems use a hierarchical planner followed by a separately learned low-level arm–hand controller [16]. In contrast, our shared VLA predicts arm poses and hand-pose representations in the same action chunk without a separately learned low-level hand controller. We refer to this as end-to-end arm–hand action prediction; Hand Intent selects among the predicted hand poses for execution.

The main obstacle is the size and structure of the hand action space. Useful hand motions follow coordinated patterns in the 16-dimensional joint space [17–21]. We address two bottlenecks. First, **Hand Synergy Latent** uses a shared PCA basis to represent 16-dimensional commands in four dimensions. We integrate this representation as the hand-action space of a chunk-predicting arm–hand VLA. Second, **Hand Intent** predicts target hand progress from the current scene and robot state and selects a timing-consistent hand action from the same VLA chunk.

We evaluate grasping, pushing with the distal index finger, and pulling. We collected 300 Apple Vision Pro demonstrations (100 per task) and organized them in an Open X-Embodiment-compatible format for VLA fine-tuning. Fig. 1 summarizes the common pipeline used to compare Direct16, Synergy4 with fixed selection, and Synergy4+Intent. The contributions are:

- Integration of a task-shared four-dimensional PCA hand-action interface into a chunk-predicting arm–hand VLA, with decoding to 16-dimensional joint commands at execution.
- Observation-conditioned hand-action selection from each VLA action chunk while keeping the arm chunk step fixed.
- Repeated real-robot evaluations that quantify the effects of compact hand-action representation and intent-guided selection under nominal-position conditions and evaluate A2 task completion at unseen initial positions.

2. RELATED WORK

2.1. Vision-Language-Action Policies

Large-scale robot learning has increasingly moved toward generalist policies that condition on images, robot

Minchan Jin and Geunha Kim are with the Department of Artificial Intelligence, Hanyang University, 222 Wangsimni-ro, Seongdong-gu, Seoul 04763, Korea (e-mails: qosehf2001@hanyang.ac.kr, rlarmsgk2@hanyang.ac.kr). Jun Moon is with the Department of Electrical Engineering, Hanyang University, 222 Wangsimni-ro, Seongdong-gu, Seoul 04763, Korea (e-mail: junmoon@hanyang.ac.kr).
† Equal contribution. * Corresponding author.

©ICROS, KIEE and Springer 2026

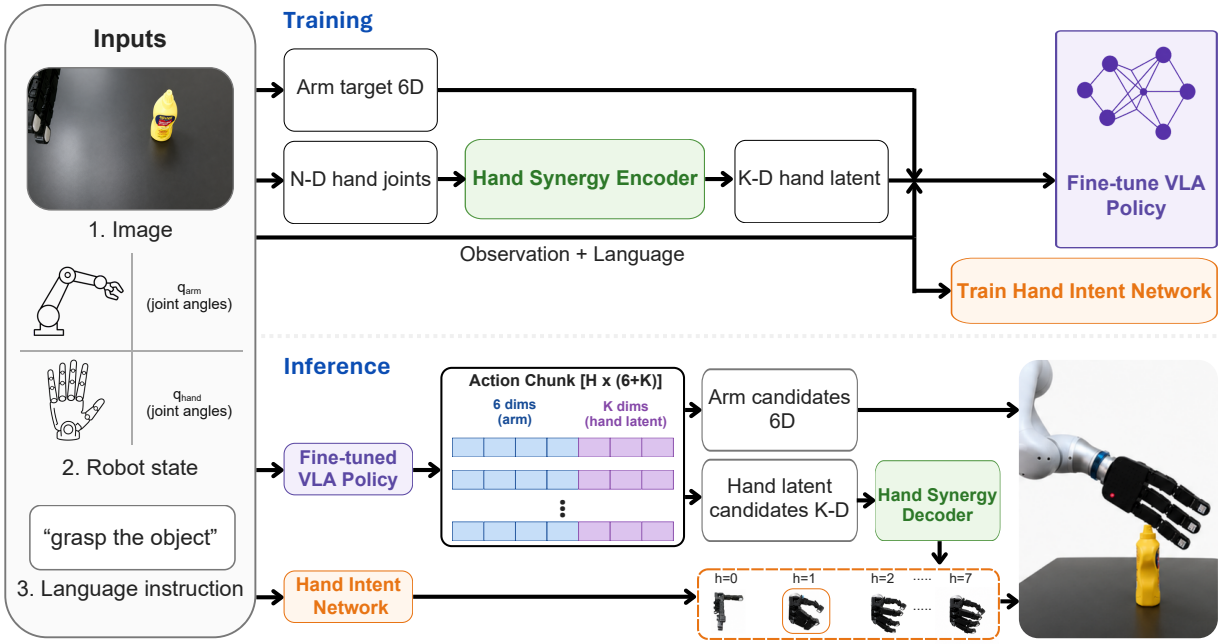


Fig. 1. Overall training and inference pipeline. During training, demonstrations are converted into compact Hand Synergy Latent action labels for VLA fine-tuning, and the same demonstrations supervise Hand Intent. During inference, the VLA predicts an arm–hand action chunk, the hand candidates are decoded to full joint commands, and Hand Intent selects a candidate for execution. The illustrated hand candidates are temporal steps within one task-conditioned action chunk.

states, and language instructions. RT-series models, SayCan, Octo, and OpenVLA show that robot policies can benefit from broad data, reusable architectures, and language-conditioned fine-tuning [1, 3–5, 22]. Embodied multimodal models and robot imitation policies further show that visual-language representations can be connected to low-level robot actions, enabling policies to interpret instructions, objects, and manipulation contexts in a unified interface [6–8]. Multi-task robot datasets also show that broad manipulation data can improve generalization across objects, instructions, and environments [2, 23, 24].

Several imitation-learning policies predict chunks of future actions to improve temporal consistency and reduce compounding errors [25–28]. Octo likewise supports chunked action prediction, allowing an executor to choose among future offsets from one policy output [1]. In dexterous manipulation, these offsets may span open, transitioning, and task-specific hand postures, so the arm and hand need not execute the same chunk step.

2.2. Dexterous Manipulation

Dexterous manipulation requires coordinated multi-joint control, contact-rich interactions, and precise timing. Prior work has explored reinforcement learning, imitation learning, and sim-to-real transfer for multi-fingered hands [12–14]. Benchmark environments and datasets show that dexterous interaction depends on both action dimensionality and the geometry of the hand, object, and task [15]. Recent IJCAS work on GASP introduces a generative adversarial state-policy and multimodal sensing framework for dexterous manipulation, emphasizing state guidance and sensory feedback [29]. Recent dexterous VLA systems connect language-conditioned perception to dexterous actions, often through hierarchical structures or separate hand-control components [16].

2.3. Hand Synergies

Human hand studies show that natural postures are not arbitrary points in a high-dimensional joint space but often lie in low-dimensional subspaces known as hand synergies [17]. Robotic grasping studies similarly show that posture subspaces reduce the complexity of planning and control while preserving useful coordinated configurations [18]. This structure is useful for VLA policies because it reduces the dimension of the predicted hand action.

Building on established synergy and posture-subspace methods, we use a shared low-dimensional PCA representation as the VLA hand-action space and decode it to full joint commands at execution. We pair this representation with observation-conditioned hand-action selection from a VLA-predicted chunk. The shared basis provides a common hand-action space across the three tasks, while task-dependent hand motions are retained through the de-

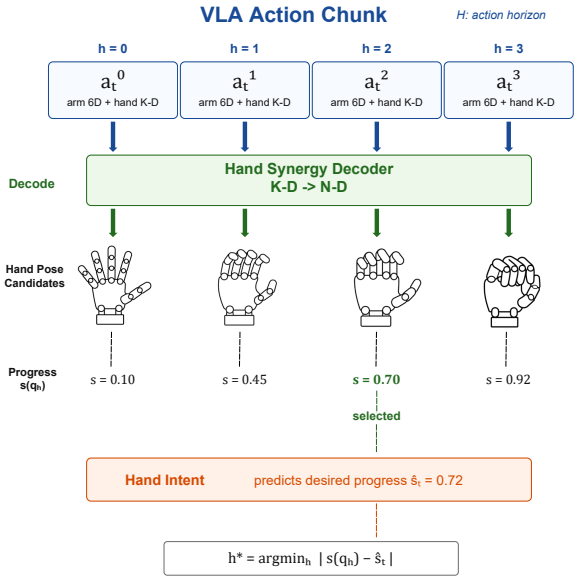


Fig. 2. Action chunk execution with intent-guided hand candidate selection. Four candidate steps are shown schematically; the implementation evaluates all eight steps, $h \in \{0, \ldots, 7\}$.

coded joint commands.

Linear posture coordinates are established in hand-synergy and eigengrasp models, where coordinated hand postures are parameterized along directions in joint space [17, 18]. Prior synergy methods primarily use these low-dimensional subspaces to describe, reconstruct, or search hand postures, whereas action-chunking policies predict temporally extended action sequences [1, 25–28]. Hierarchical dexterous VLA approaches may additionally rely on a separately learned low-level arm–hand controller [16]. Our method instead uses the synergy representation as the hand-action interface of a chunk-predicting arm–hand VLA and selects only the hand candidate from the jointly predicted chunk using the current observation, while keeping the task-specific arm step fixed.

3. METHOD

3.1. Overall Pipeline

Our system combines an Octo VLA, a fixed PCA decoder, and a Hand Intent selector. During training, each demonstration provides an RGB observation, a language instruction, the robot state, and an arm–hand action. The arm action is an absolute six-dimensional tool center point (TCP) pose, and the hand command is encoded as a synergy latent. Octo predicts an eight-step chunk containing both arm poses and hand latents. We refer to this joint prediction as end-to-end arm–hand action prediction.

At execution, the PCA decoder reconstructs each hand latent as a 16-dimensional Allegro command. The arm uses a task-specific fixed chunk step, while Hand Intent selects the hand step from the Octo-predicted chunk. Hand Intent does not generate a new hand command; it selects among the hand commands already predicted by Octo. This allows the arm and hand to use different temporal offsets from the same action chunk. Because selection operates on decoded candidates from a predicted chunk, the structure is, in principle, compatible with VLA backbones that output action chunks.

3.2. Hand Synergy Latent

Let $\mathbf{q}^{\text{hand}} \in \mathbb{R}^{16}$ denote an Allegro hand command in radians. We fit its mean $\boldsymbol{\mu} \in \mathbb{R}^{16}$ and PCA basis $\mathbf{B}_K \in \mathbb{R}^{16\times K}$ from the 270 training demonstrations; K is the number of retained components. The subscript t denotes timestep, and a hat denotes prediction or reconstruction. Joint angles are mean-centered without per-joint standardization. The same fixed basis is used for grasp, push, and pull, yielding a task-shared, Allegro-specific action basis.

Each policy action is

$$\mathbf{a}_t = [x_t, y_t, z_t, \phi_t, \theta_t, \psi_t, \mathbf{z}_t^{\text{hand}}], \tag{1}$$

Here, $\mathbf{a}_t$ is the policy action; the first two triplets are the TCP position and orientation coordinates, and $\mathbf{z}_t^{\text{hand}} \in \mathbb{R}^K$ is the hand latent. Encoding and decoding are

$$\mathbf{z}^{\text{hand}} = \mathbf{B}_K^\top(\mathbf{q}^{\text{hand}} - \boldsymbol{\mu}), \tag{2}$$

$$\hat{\mathbf{q}}^{\text{hand}} = \boldsymbol{\mu} + \mathbf{B}_K \mathbf{z}^{\text{hand}}. \tag{3}$$

We use $K = 4$, reducing the policy output from 22 to 10 dimensions. Before execution, direct 16-dimensional predictions and PCA-decoded commands are clipped to the joint ranges observed in the training data.

3.3. Hand Intent

Successful contact requires the hand transition to match the arm–object interaction; an early or late transition can displace the object or miss contact. Hand Intent uses the current observation to select the decoded VLA hand candidate whose progress matches the required contact stage while retaining the fixed arm step.

Let $\mathbf{q}_{\text{open}}$ denote the open-hand pose and $\mathbf{q}_{\text{task}}$ the fixed progress reference associated with the current task-specific Hand Intent checkpoint, as detailed in Section 4. With $\mathbf{d} = \mathbf{q}_{\text{task}} - \mathbf{q}_{\text{open}}$, we define normalized hand progress as

$$s(\mathbf{q}) = \text{clip}\left(\frac{(\mathbf{q} - \mathbf{q}_{\text{open}})^\top \mathbf{d}}{\|\mathbf{d}\|^2}, 0, 1\right). \tag{4}$$

This projection provides a scalar measure of the open-to-target transition, with $\mathrm{clip}(u,0,1)$ restricting u to $[0,1]$. Here, linearity refers only to the coordinate direction in joint space; it does not constrain the demonstrated or executed hand trajectory to follow the line segment. The progress value orders a pose along the open-to-target direction, while off-direction finger coordination is quantified by the residual below. For a demonstrated command $\mathbf{q}_t^{\mathrm{demo}}$, the projected pose and normalized residual are

$$\mathbf{q}_t^{\mathrm{line}} = \mathbf{q}_{\mathrm{open}} + s(\mathbf{q}_t^{\mathrm{demo}})\mathbf{d}, \tag{5}$$

$$r_t = \frac{\|\mathbf{q}_t^{\mathrm{demo}} - \mathbf{q}_t^{\mathrm{line}}\|_2}{\|\mathbf{d}\|_2 + \varepsilon}. \tag{6}$$

The residual distinguishes hand configurations with similar scalar progress but different finger geometry. Here, $\|\cdot\|_2$ is the Euclidean norm and $\varepsilon > 0$ prevents division by zero.

Hand Intent receives a 64×64 RGB crop $\mathbf{I}_t$ and the observed robot state $\mathbf{x}_t$. The image encoder is a three-layer CNN with 16, 32, and 64 channels, followed by global average pooling into a 64-dimensional feature. A layer-normalized MLP produces a 128-dimensional state feature. A two-layer fusion head with hidden size 128 and dropout 0.05 combines both features. Three output heads predict progress $\hat{s}_t$, a nonnegative residual $\hat{r}_t$, and two phase logits $\hat{\mathbf{p}}_t$:

$$(\hat{s}_t, \hat{r}_t, \hat{\mathbf{p}}_t) = f_{\boldsymbol{\omega}}(\mathbf{I}_t, \mathbf{x}_t). \tag{7}$$

Here, $\boldsymbol{\omega}$ denotes the learned network parameters. For all tasks, the progress and residual heads use sigmoid and Softplus, respectively, and the phase head outputs logits. Training minimizes

$$\mathcal{L} = \lambda_s \mathcal{L}_{\mathrm{prog}} + \lambda_r \mathcal{L}_{\mathrm{res}} + \lambda_p \mathcal{L}_{\mathrm{phase}}, \tag{8}$$

Progress and residual use mean-squared error against $s(\mathbf{q}_t^{\mathrm{demo}})$ and the training-95th-percentile-normalized r_t, respectively. Phase uses cross-entropy with class 0 below $s = 0.5$ and class 1 otherwise. Their weights are 1.0, 0.15, and 0.5. Demonstrations are grouped by the task in their language instructions, and one identically configured Hand Intent checkpoint is trained per task. Residual and phase serve as training-only auxiliary targets.

For every decoded hand action, the selector computes progress analytically and chooses the chunk step

$$h^* = \arg\min_{h \in \{0,\ldots,7\}} \left| s(\hat{\mathbf{q}}_{t,h}^{\mathrm{hand}}) - \hat{s}_t \right|. \tag{9}$$

Here, h is the zero-based chunk index, $\hat{\mathbf{q}}_{t,h}^{\mathrm{hand}}$ its decoded hand candidate, and h^* the selected step. Figure 2 illustrates the hand-candidate selection rule. We use the following architecture labels throughout the experiments:

- **Direct16 (A0):** Octo directly predicts a 6-dimensional TCP pose and a 16-dimensional hand command at each chunk step. The arm and hand execute the same task-specific fixed step.
- **Synergy4 (A1):** Octo predicts a 6-dimensional TCP pose and a 4-dimensional hand latent at each step, and the fixed PCA decoder reconstructs the 16-dimensional hand command. The arm and hand use the same fixed steps as A0.
- **Intent (A2):** The A1 VLA checkpoint and its fixed arm step are retained, while Hand Intent selects only the hand step from the eight decoded candidates using Eq. (9).

The task-specific fixed steps used by A0 and A1, and by the A2 arm, are listed in Table 1. We selected them for repeatability in preliminary robot trials and fixed them before the reported evaluations.

4. SETUPS

4.1. Tasks, Robot Setup, and Policy Inputs

We evaluate three tasks with different hand–object contacts. In grasp, the hand encloses the object and lifts it from the table. In push, the index fingertip topples the upright object in the instructed direction while the hand maintains a pinch-like posture. In pull, the flexed index finger engages behind the object and pulls it until it topples. These tasks cover whole-hand grasping, fingertip pushing, and pulling. The evaluated setup uses one object identity and one instruction per task.

We set the open-hand reference to $\mathbf{q}_{\mathrm{open}} = \mathbf{0}_{16}$, meaning that all 16 Allegro joint coordinates are 0 rad in the joint convention used by the controller and dataset. For each task k, we take the final observed 16-dimensional Allegro joint-position vector from each of its 90 training demonstrations and use the arithmetic mean of these vectors as $\mathbf{q}_{\mathrm{task}}^k$. We fix these reference poses before Hand Intent training and evaluation. The task specified in the language instruction routes the current observation to the corresponding task-specific Hand Intent checkpoint, which uses its fixed $\mathbf{q}_{\mathrm{task}}^k$ to define hand progress; the target pose is not predicted from the instruction. Our setup uses a Doosan A0509 arm, a right Allegro hand, and an Intel RealSense D455 RGB camera. For compatibility with the Open X-Embodiment training interface, a fixed workspace crop is resized to 256×256 for the VLA.

The 44-dimensional execution state is

$$\mathbf{x}_t = [\mathbf{x}_6^{\mathrm{TCP}}, \mathbf{q}_6^{\mathrm{arm}}, \mathbf{q}_{16}^{\mathrm{hand}}, \dot{\mathbf{q}}_{16}^{\mathrm{hand}}], \tag{10}$$

The state components are normalized using statistics from the training demonstrations, and the language instruction is supplied separately to the VLA. The policy predicts an absolute TCP target in the robot base frame, which is converted to the controller format and executed through inverse kinematics.

Table 1. Training and execution settings. Fixed-step values are ordered as grasp, push, and pull.

Setting	Value
VLA backbone	Octo-base-1.5
Optimizer / learning rate	AdamW / 3×10^{-5}
Batch size / warmup	16 / 150 steps
Observation frames / chunk length	2 / 8
Hand Intent optimizer / LR	AdamW / 10^{-3}
Hand Intent batch / hidden dimension	128 / 128
A0/A1 fixed arm–hand step h	5, 5, 1
A2 fixed arm step h	5, 5, 1

4.2. Demonstration Collection and Dataset

We collected demonstrations using an Apple Vision Pro while directly viewing the robot and workspace. The headset provided the right-wrist pose and hand landmarks. At the start of each demonstration, we recorded the first valid right-wrist pose and the current robot TCP pose as a reference pair. Subsequent wrist translation was mapped relative to this pair as $[\Delta y, -\Delta x, \Delta z]$, and the relative wrist rotation was applied to the reference TCP orientation. We generated the corresponding arm targets through inverse kinematics.

We used DexPilot with the Allegro URDF to retarget the hand landmarks to the 16 Allegro joints and sent smoothed joint commands to the hand controller. The commanded hand joints are the 16-dimensional targets sent to the controller, while the observed robot state contains the measured arm and hand joint positions. We recorded synchronized RGB images, observed robot states, arm TCP targets, commanded hand joints, task labels, and instructions at 10 Hz. Each training example follows the observation and chunk settings in Table 1; at this sampling rate, chunk step h represents a target $0.1h$ s after the observation.

The data are organized in an Open X-Embodiment-compatible format so that the VLA can be fine-tuned with image observations, robot states, language instructions, and action chunks.

We collected 300 demonstrations equally across the three tasks. Each demonstration includes a language instruction specifying its task; these task assignments are used to group demonstrations for Hand Intent training. We reserve ten episodes per task for validation and use the remainder for training. The PCA and VAE models, per-joint action bounds, and normalization statistics use only the training demonstrations.

5. EXPERIMENTS

We organize the experiments around eight evaluation objectives:

- **Latent-Dimension Selection:** Select K using reconstruction error and cumulative explained variance.
- **Equal-Dimensional PCA–VAE Comparison:** Compare PCA-K4 with the equal-dimensional VAE-K4 baseline.
- **Hand-Progress Alignment:** Compare A1 and A2 transition timing and pose progress over repeated robot trials.
- **Nominal-Position Task Outcomes:** Compare A0, A1, and A2 task outcomes over 90 real-robot trials.
- **Generalization to Unseen Initial Positions:** Evaluate A2 at 21 unseen initial positions per task across three independently trained VLA checkpoints.
- **Action-Representation Learning Speed:** Compare the convergence of A0 and A1 across six training seeds.
- **Deployed Control-Loop Runtime:** Measure Hand Intent latency and the achieved full control-loop frequency.
- **Hand Intent Supervision Ablation:** Isolate the effects of progress, residual, and phase supervision across ten training seeds.

5.1. Latent-Dimension Selection

Figure 3 compares reconstruction error and explained variance across latent dimensions. Cumulative explained variance is computed from the training demonstrations, and reconstruction errors are computed on the fixed validation split. Both measures improve sharply through $K = 4$, where the overall and task-reference errors are already small. The marginal gains at higher dimensions support the use of $K = 4$ as the hand-action dimension.

5.2. Equal-Dimensional PCA–VAE Comparison

VAE-K4 uses a two-layer encoder with 128 and 64 hidden units, four-dimensional mean/log-variance heads, and a mirrored decoder [30]. It is trained for 300 epochs on the same training demonstrations using reconstruction loss and a KL term with $\beta = 10^{-3}$; the deterministic posterior mean is used as the action target. VAE-K4 achieves lower RMSE and p95 reconstruction errors, whereas PCA-K4 reaches both learning criteria earlier (Table 2). The crossing steps use the normalized convergence criteria defined in Section 5.6.

5.3. Hand-Progress Alignment

Let τ_k^{demo} be the mean 0.5-progress transition time across the training demonstrations for task k, and let τ_i^{exec} be the corresponding transition time in trial i. Executed progress is computed from the observed Allegro joint positions. The timing error is

$$E_i^{\text{time}} = 100 \frac{|\tau_i^{\text{exec}} - \tau_k^{\text{demo}}|}{\tau_k^{\text{demo}}}, \tag{11}$$

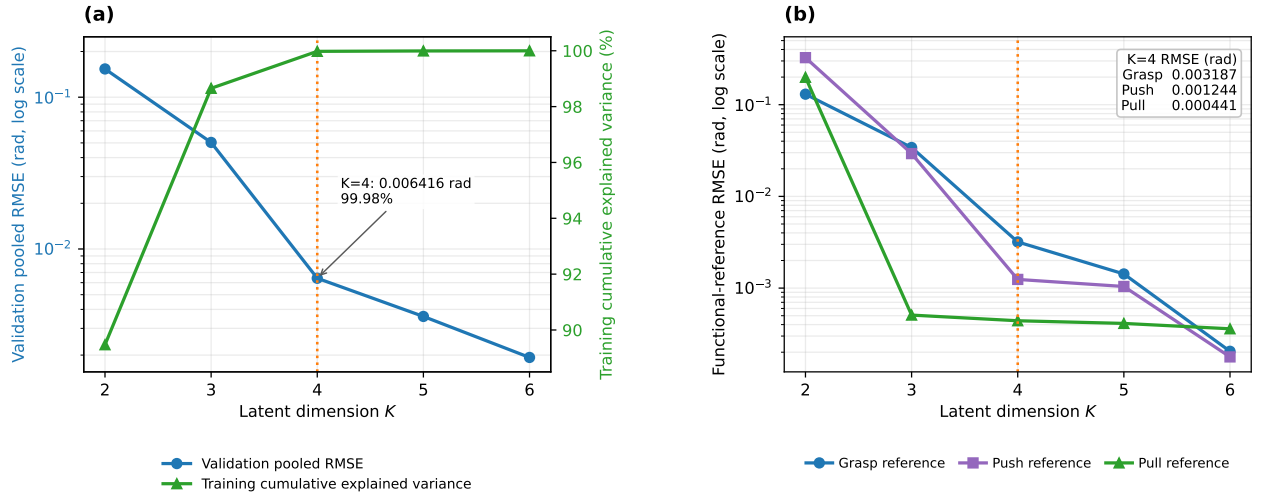


Fig. 3. Selection of Hand Synergy dimension K. (a) RMSE of scalar joint errors pooled over all validation frames and joints (left axis) and cumulative explained variance from the training-set PCA fit (right axis). (b) RMSE across 16 joints for the grasp, push, and pull reference poses. $K = 4$ is marked in both panels.

Table 2. Equal-dimensional PCA/VAE comparison. MAE, RMSE, and p95 pool scalar joint errors over all frames and joints in the fixed validation split. Crossing steps are mean±sample SD over six seeds; their reduction is for PCA-K4 relative to VAE-K4. Lower reconstruction errors and crossing steps are better, whereas larger reductions indicate greater improvement.

Measure	PCA-K4	VAE-K4	Reduction
MAE (10^{-3} rad)	2.727	2.845	–
RMSE (10^{-3} rad)	6.416	4.600	–
p95 (10^{-3} rad)	11.399	8.782	–
90% step	1066.7 ± 60.6	1266.7 ± 103.3	15.8%
95% step	1591.7 ± 73.6	1933.3 ± 121.1	17.7%

The pose-progress error compares the executed trajectory with the mean demonstration trajectory $\bar{s}_k^{\mathrm{demo}}(t)$ in physical elapsed time:

$$E_i^{\mathrm{pose}} = \frac{100}{N_i} \sum_{n=1}^{N_i} \left| s_i^{\mathrm{exec}}(t_{i,n}) - \bar{s}_k^{\mathrm{demo}}(t_{i,n}) \right|, \tag{12}$$

where N_i is the number of recorded timestamps. Table 3 reports the mean and sample standard deviation over ten trials per task and condition.

A2 reduces all alignment errors in Table 3, indicating closer agreement with the demonstration progress trajectories. Figure 4 provides a representative view of hand-progress alignment around the demonstration transition, while Table 3 quantifies the repeated A1–A2 errors in physical elapsed time.

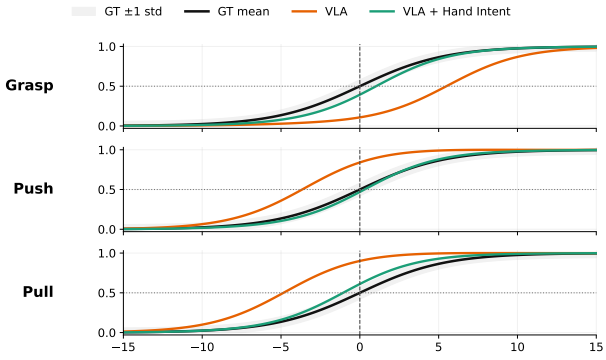


Fig. 4. Representative normalized hand-progress curves for A1 and A2, aligned to the demonstration transition. The horizontal axis shows relative timesteps, with zero at the demonstration transition. Shading denotes mean±standard deviation of the demonstration reference trajectories; Table 3 uses physical elapsed time from episode start.

5.4. Nominal-Position Task Outcomes

We evaluated A0, A1, and A2 in ten trials per task and condition at the same nominal object position. A0 and A1 compare separately trained policies under matched data and settings, whereas A1 and A2 share the same VLA checkpoint and differ only in hand-candidate selection. Table 4 reports complete, partial, and failed outcomes. Complete outcomes correspond to a stable multi-finger lift for grasp, fingertip-directed toppling for push, and behind-object engagement followed by pulling for pull. Partial outcomes achieve the final task goal but do not satisfy the intended contact pattern or execution-quality criterion,

Table 3. Hand-progress alignment errors for A1 and A2. Values are mean±sample SD over ten trials per task; lower errors are better, and reduction is $100(1-\bar{E}_{A2}/\bar{E}_{A1})$.

Task	Metric	A1 ($n=10$)	A2 ($n=10$)	Reduction
Grasp	Absolute timing error	$5.5\pm1.1\%$	$1.2\pm0.4\%$	78.2%
	Pose-progress error	$2.2\pm0.5\%$	$1.1\pm0.3\%$	50.0%
Push	Absolute timing error	$3.5\pm0.9\%$	$0.5\pm0.2\%$	85.7%
	Pose-progress error	$2.7\pm0.6\%$	$1.5\pm0.4\%$	44.4%
Pull	Absolute timing error	$4.8\pm1.1\%$	$1.0\pm0.3\%$	79.2%
	Pose-progress error	$1.9\pm0.4\%$	$0.9\pm0.3\%$	52.6%

Table 4. Nominal-position outcomes over ten trials per task and condition. The Complete, Partial, and Failure columns report outcome counts; the final column gives the strict complete rate and its two-sided 95% Wilson score interval.

Task	Condition	Complete	Partial	Failure	Strict complete rate, % [95% Wilson CI]
Grasp	A0	6	4	0	60.0 [31.3, 83.2]
	A1	6	4	0	60.0 [31.3, 83.2]
	A2	**10**	0	0	**100.0 [72.2, 100.0]**
Push	A0	6	1	3	60.0 [31.3, 83.2]
	A1	9	0	1	90.0 [59.6, 98.2]
	A2	**10**	0	0	**100.0 [72.2, 100.0]**
Pull	A0	0	0	10	0.0 [0.0, 27.8]
	A1	2	8	0	20.0 [5.7, 51.0]
	A2	**10**	0	0	**100.0 [72.2, 100.0]**

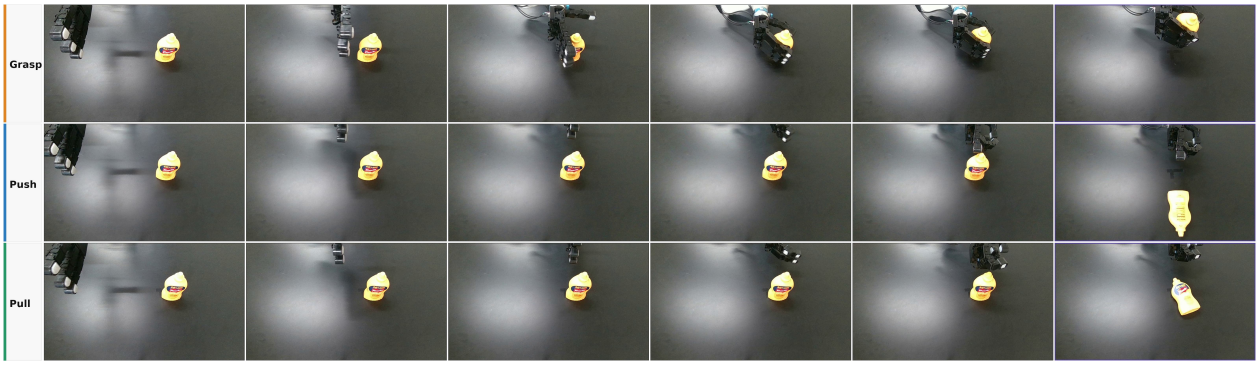


Fig. 5. Representative successful A2 nominal-position rollouts for grasp, push, and pull. The columns show the progression from approach to final task completion.

while failures do not achieve the final task goal. For the strict complete rate, complete is coded as success, whereas partial and failure are combined as non-complete. We report a two-sided 95% Wilson score interval because each condition contains ten trials and several observed rates are near 0 or 1.

Table 4 shows that A1 improves hand–object engagement over A0, particularly for push and pull. A2 achieves 10/10 complete outcomes for every task, whereas A1 retains partial or failed outcomes. For pull, a two-sided Fisher exact test comparing strict complete outcomes against partial/failure outcomes gives $p_{adj} = 0.00429$ after Holm correction across the six task-wise A0–A1 and A1–A2 comparisons.

Figure 5 shows the temporal progression of the three nominal-position tasks using representative successful A2 executions. Table 4 reports the classified outcomes, and the accompanying video provides the complete contact sequences.

Qualitative inspection of the pull executions indicated that some A1 trials pressed the object, whereas representative A2 executions placed the index finger behind the object before pulling. The accompanying video illustrates these contact patterns from pre-contact to the final outcome.

Representative real-robot executions are provided in

Table 5. A2 binary task-completion rate at unseen initial positions. Completed trials aggregate three independently trained VLA checkpoints evaluated on the same 21 task-specific positions. SD is computed across the three checkpoint-level completion rates in percentage points.

Task	Completed trials	Rate (%)	SD (pp)
Grasp	63/63	100.00	0.00
Push	42/63	66.67	4.76
Pull	35/63	55.56	2.75

Online Resource 1.

5.5. Generalization to Unseen Initial Positions

As a controlled first step, we varied only the initial object position because this changes the robot approach and hand–object contact geometry while keeping the object identity, background, instruction, and robot platform fixed. For each task, we manually placed the object at 21 distinct, randomly selected positions that were not included in the training demonstrations. We evaluated A2 using three independently trained VLA checkpoints on the same task-specific positions and the same fixed Hand Intent checkpoint for that task, yielding 63 trials per task and 189 trials in total. The position sets were selected separately because the feasible approach and contact regions differed across the three task motions.

Tables 4 and 5 serve different evaluation purposes. Table 4 analyzes execution quality at the nominal position and therefore distinguishes complete, partial, and failed outcomes. Both complete and partial outcomes achieve the final task goal, but partial outcomes do not satisfy the intended contact pattern or execution-quality criterion. Table 5 evaluates generalization to unseen initial positions using the same binary final task-goal criterion for every task: complete and partial outcomes are combined as task completion, whereas failure is counted as non-completion. The binary task-completion rates in Table 5 are therefore not intended for direct comparison with the strict complete rates in Table 4.

Figure 6 presents representative challenging cases observed during the A2 unseen-position evaluation. Together with the binary task-completion rates in Table 5, these examples show that A2 can reach the final task goals under position variation, although completion and execution quality remain task-dependent.

5.6. Action-Representation Learning Speed

A0 and A1 are trained separately under the same experimental conditions and differ only in the hand-action representation.

To compare learning speed across action representations, we normalize the best validation loss within each run. Let t_0 and T be the first and final evaluated optimization steps and $L_t^{\min} = \min_{t_0 \le \tau \le t} L_\tau$. We define

$$c_t = \frac{L_{t_0}^{\min} - L_t^{\min}}{L_{t_0}^{\min} - L_T^{\min}}. \tag{13}$$

We evaluate validation loss every 100 steps and linearly interpolate the first crossings of $c_t = 0.90$ and $c_t = 0.95$. These within-run criteria summarize learning speed across representations.

A1 reaches both criteria earlier than A0. It requires 13.5% fewer steps at the 90% criterion and 28.2% fewer steps at the stricter 95% criterion (Fig. 7).

Table 6. Runtime latency. The reported statistic is identified for each measurement. Lower latency is better; the measured full loop operates at 4.874 ± 0.010 Hz.

Measurement	Statistic	Value (ms)	p95 (ms)
Intent network only	Mean	0.556	0.627
Preprocess through selection	Mean	3.884	4.604
Intent path in deployed loop	Mean	4.029	7.261
Complete control cycle	Median	205.17	210.01

5.7. Deployed Control-Loop Runtime

Table 6 reports four nested measurement scopes. “Intent network only” measures the forward pass from preprocessed tensors. “Preprocess through selection” additionally includes image resizing, state normalization, Hand Intent inference, analytical progress computation, and candidate selection. “Intent path in deployed loop” measures this path inside the robot process, including in-loop scheduling overhead. “Complete control cycle” covers image acquisition, state preprocessing, VLA inference, Hand Intent selection, command conversion, and robot communication. The first two scopes use batch size one and one CPU thread on the deployment workstation, with 20 warm-up calls followed by 200 timed calls. The deployed Intent path is measured over 1,800 active control steps, and the complete-cycle frequency is summarized over 15 runtime trials. The first three rows report mean and p95 latency; the complete cycle reports median and p95 because scheduling and I/O variation affect the end-to-end period. The isolated network requires 0.556 ms, and the Intent path in the deployed loop requires 4.029 ms on average, approximately 2% of the 205.17-ms control cycle. The measured 4.874 ± 0.010 Hz is the achieved end-to-end robot operating frequency, rather than the neural-network inference rate or the 10-Hz training-data rate.

5.8. Hand Intent Supervision Ablation

The projection provides a scalar measure of hand progress. Let $s_t^{\text{demo}} = s(\mathbf{q}_t^{\text{demo}})$ denote demonstration-derived progress. Across all tasks, residuals are larger

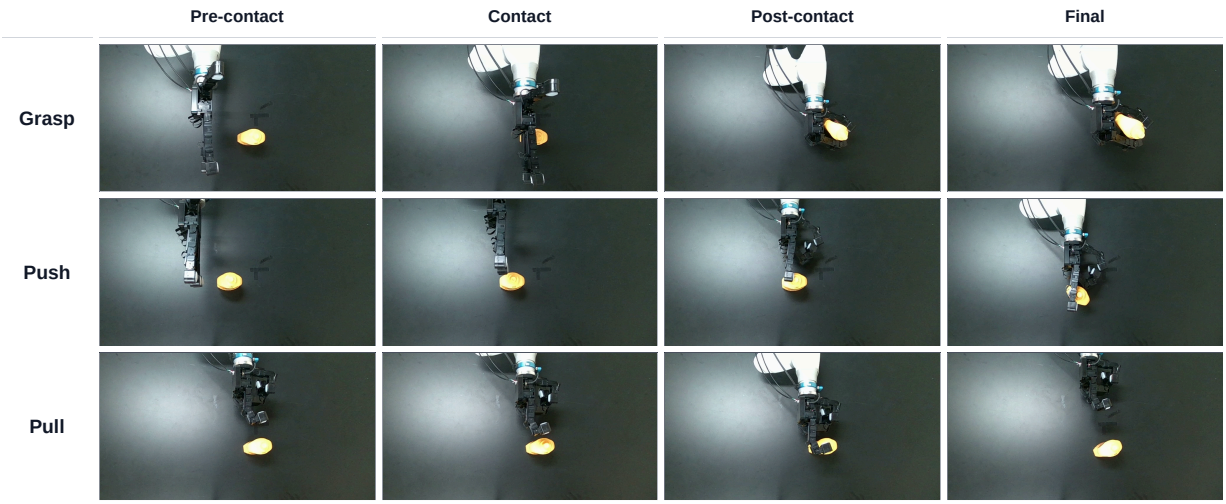


Fig. 6. Representative challenging cases from the A2 unseen-position evaluation. The grasp sequence satisfies the binary task-completion criterion because the object is lifted, but exhibits marginal execution quality because it is not lifted in the intended stable orientation. The push and pull sequences do not achieve their final task goals. The columns show the sequence from pre-contact to the final observed outcome.

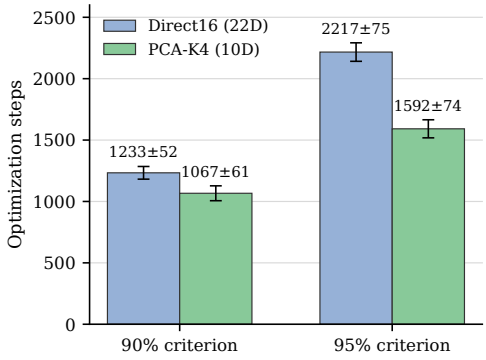


Fig. 7. A0–A1 learning-speed comparison. Bars show the mean crossing step over six training seeds, error bars show sample SD, and lower values indicate earlier convergence.

within the transition window than over the full validation trajectories, with transition p95 values ranging from 0.291 to 0.363. Thus, the scalar coordinate supplies an ordering of progress rather than a linear reconstruction of the full hand trajectory. These transition residuals motivate the auxiliary residual target for deviations from the one-dimensional reference.

A transition frame is correct when $|\hat{s}_t - s_t^{\text{demo}}| \leq 0.1$. On the fixed validation split, evaluation is restricted to $0.05 < s_t^{\text{demo}} < 0.95$. Within each seed, we pool frames by task and macro-average the task accuracies. Table 7 summarizes the independent seed sets, each containing one checkpoint per task.

Table 7. Hand Intent transition accuracy. Values are task-macro mean±sample SD over ten training seeds on the fixed validation split; higher values are better.

Supervision	Accuracy (%)
Progress only	70.8 ± 3.1
Progress + residual	77.3 ± 2.6
Progress + phase	81.2 ± 2.1
Full	86.7 ± 1.6

All Hand Intent variants use predicted progress for candidate selection; residual and phase are auxiliary training targets. A1 uses a task-specific fixed candidate, whereas A2 selects a hand candidate from the same VLA chunk using the current observation.

6. CONCLUSION

We presented an end-to-end arm–hand action-prediction pipeline evaluated through the A0, A1, and A2 configurations. A1 reaches the model-specific normalized convergence criteria earlier than A0, while A2 reduces hand-progress alignment errors and produces complete rather than partial pull outcomes at the nominal position. Across the tested unseen positions and checkpoints, grasp completed 63/63 trials, whereas push and pull completed 42/63 and 35/63 trials, respectively.

The unseen-position evaluation varies only the initial object position and uses Octo with task-specific target poses and Hand Intent models. The shared PCA basis

is learned from all three evaluated tasks in the Allegro joint space, so the present evidence concerns reuse across their contact patterns within one hand morphology. Future work will evaluate transfer to held-out tasks, objects, and robotic hands. For hands with different joint spaces or kinematics, we will investigate both hand-specific basis refitting and a common retargeted latent representation. This evaluation will also cover unseen scenes and instructions, additional VLA backbones and selector or controller designs, and instrumented object-pose measurements.

DECLARATIONS

Conflict of Interest

The authors declare that there are no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Authors' Contributions

Minchan Jin developed the robot data collection and inference pipeline, conducted the experiments, and drafted the manuscript. Geunha Kim contributed to the experimental system, data collection, and manuscript preparation. Jun Moon supervised the research and reviewed the manuscript.

Funding

This work was supported in by the Korea Institute of Energy Technology Evaluation and Planning(KETEP) and the Ministry of Climate, Energy Environment(MCEE) of the Republic of Korea (No. RS-2024-00422103, EV Smart Charging Platform Innovation Research Center), in part by the Korea Institute of Energy Technology Evaluation and Planning(KETEP) and the Ministry of Climate, Energy Environment(MCEE) of the Republic of Korea (NO. RS-2026-25530097), in part by the Institute of Information communications Technology Planning and Evaluation (IITP) grant funded by the Korea government (Ministry of Science and ICT, MSIT) (RS-2020-II201373), in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00397400), in part by the National Research Foundation (NRF) funded by the Korean government (MSIT) (No. RS-2024-00421129), and in part by the Technology Innovation Program (20018112) funded by the Ministry of Trade, Industry and Energy (MOTIE, Korea).

Data Availability

The data and code generated in this study are available from the corresponding author upon reasonable request. Online Resource 1 provides representative real-robot executions for the nominal-position comparison and unseen-position generalization evaluation of grasp, push, and pull.

REFERENCES

[1] Octo Model Team, "Octo: An open-source generalist robot policy," arXiv preprint arXiv:2405.12213, 2024.

[2] Open X-Embodiment Collaboration, "Open X-Embodiment: Robotic learning datasets and RT-X models," arXiv preprint arXiv:2310.08864, 2023.

[3] A. Brohan et al., "RT-1: Robotics transformer for real-world control at scale," arXiv preprint arXiv:2212.06817, 2022.

[4] B. Zitkovich et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. Conference on Robot Learning*, pp. 2165–2183, 2023.

[5] M. J. Kim et al., "OpenVLA: An open-source vision-language-action model," arXiv preprint arXiv:2406.09246, 2024.

[6] D. Driess et al., "PaLM-E: An embodied multimodal language model," arXiv preprint arXiv:2303.03378, 2023.

[7] X. Li et al., "Vision-language foundation models as effective robot imitators," arXiv preprint arXiv:2311.01378, 2023.

[8] K. Black et al., "π_0: A vision-language-action flow model for general robot control," arXiv preprint arXiv:2410.24164, 2024.

[9] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn, "BC-Z: Zero-shot task generalization with robotic imitation learning," in *Proc. Conference on Robot Learning*, pp. 991–1002, 2022.

[10] M. Shridhar, L. Manuelli, and D. Fox, "CLIPort: What and where pathways for robotic manipulation," in *Proc. Conference on Robot Learning*, pp. 894–906, 2022.

[11] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, "VIMA: General robot manipulation with multimodal prompts," arXiv preprint arXiv:2210.03094, 2022.

[12] A. Rajeswaran, V. Kumar, A. Gupta, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," in *Proc. Robotics: Science and Systems*, 2018.

[13] O. M. Andrychowicz et al., "Learning dexterous in-hand manipulation," *International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[14] Y. Qin, Y.-H. Wu, S. Liu, H. Jiang, R. Yang, Y. Fu, and X. Wang, "DexMV: Imitation learning for dexterous manipulation from human videos," in *Proc. European Conference on Computer Vision*, pp. 570–587, 2022.

[15] C. Bao, H. Xu, Y. Qin, and X. Wang, "DexArt: Benchmarking generalizable dexterous manipulation with articulated objects," arXiv preprint arXiv:2305.05706, 2023.

[16] Z. Zhong et al., "DexGraspVLA: A vision-language-action framework towards general dexterous grasping," in *Proc. AAAI Conference on Artificial Intelligence*, 2026.

[17] M. Santello, M. Flanders, and J. F. Soechting, "Postural hand synergies for tool use," *Journal of Neuroscience*, vol. 18, no. 23, pp. 10105–10115, 1998.

[18] M. T. Ciocarlie and P. K. Allen, "Hand posture subspaces for dexterous robotic grasping," *International Journal of Robotics Research*, vol. 28, no. 7, pp. 851–867, 2009.

[19] D. Antotsiou, G. Garcia-Hernando, and T. K. Kim, "Task-oriented hand motion retargeting for dexterous manipulation imitation," in *Proc. European Conference on Computer Vision Workshops*, pp. 287–301, 2019.

[20] A. Handa et al., "DexPilot: Vision-based teleoperation of dexterous robotic hand-arm system," in *Proc. IEEE International Conference on Robotics and Automation*, pp. 9164–9170, 2020.

[21] Y. Qin, W. Yang, B. Huang, K. Van Wyk, H. Su, X. Wang, Y.-W. Chao, and D. Fox, "AnyTeleop: A general vision-based dexterous robot arm-hand teleoperation system," arXiv preprint arXiv:2307.04577, 2023.

[22] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. Conference on Robot Learning*, pp. 287–318, 2022.

[23] H. Walke et al., "BridgeData V2: A dataset for robot learning at scale," in *Proc. Conference on Robot Learning*, 2023.

[24] A. Khazatsky et al., "DROID: A large-scale in-the-wild robot manipulation dataset," in *Proc. Robotics: Science and Systems*, 2024.

[25] A. Vaswani et al., "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems*, pp. 5998–6008, 2017.

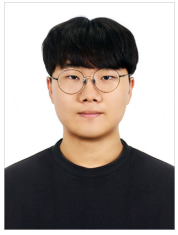
[26] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," arXiv preprint arXiv:2304.13705, 2023.

[27] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *International Journal of Robotics Research*, vol. 43, no. 9, pp. 1285–1306, 2024.

[28] S. Shridhar, L. Manuelli, and D. Fox, "PerAct: Perceiver-actor for multi-task learning of robot manipulation," in *Proc. Conference on Robot Learning*, pp. 2091–2104, 2023.

[29] Y. Go, Y. Oh, S. Kim, and J. Moon, "GASP: Generative adversarial state policy and multi-modal sensing framework for dexterous manipulation," *International Journal of Control, Automation and Systems*, pp. 1–13, 2026.

[30] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. International Conference on Learning Representations*, 2014.



Minchan Jin received the B.S. degree in electronic and IT media engineering from Seoul National University of Science and Technology, Seoul, South Korea, in 2026. He is currently pursuing an M.S. degree at Hanyang University, Seoul, Republic of Korea. His current research interests include dexterous manipulation and reinforcement learning.



Geunha Kim received the B.S. degree in multimedia engineering from Dongguk University, Seoul, Republic of Korea, in 2025. He is currently pursuing an M.S. degree at Hanyang University, Seoul, Republic of Korea. His current research interests include robotic manipulation control, reinforcement learning, and vision-language-action models.



Jun Moon (Senior Member, IEEE) received the B.S. degree in electrical and computer engineering and the M.S. degree in electrical engineering from Hanyang University, Seoul, South Korea, in 2006 and 2008, respectively, and the Ph.D. degree in electrical and computer engineering from the University of Illinois at Urbana-Champaign, USA, in 2015. He is currently an Associate Professor with the Department of Electrical Engineering, Hanyang University, Seoul, South Korea. From 2008 to 2011, he was a Researcher with the Agency for Defense Development (ADD), South Korea. From 2016 to 2019, he was an Assistant Professor with the School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology (UNIST), South Korea. From 2019 to 2020, he was an Associate Professor with the School of Electrical and Computer Engineering, University of Seoul, South Korea. His research interests include stochastic optimal control and filtering, reinforcement learning, data-driven control, distributed control, networked control systems, and mean field games. He was a recipient of the Fulbright Graduate Study Award in 2011.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.